



Gruppo SkyNet
skynetunigroup@gmail.com

Analisi dei Requisiti

Versione	<i>0.5</i>
Uso	<i>Esterno</i>
Destinatari	<i>Prof. Tullio Vardanega, Prof. Riccardo Cardin</i>
Redattori	<i>Sara Ristovic, Marco Barbiero, Dario Pirrone</i>
Verifica	<i>Dario Pirrone, Marco Barbiero</i>
Approvazione	<i>Samuele Bertin, Dario Pirrone, Andrea Fistarol</i>

Versioni del documento

Ver.	Data	Descrizione	Autore	Verificatore
0.5.0	2026/05/09	Scrittura della sottosezione "Casi d'uso trasversali"; Aggiunti 3 UC all'agente 3, la sua introduzione e gli obiettivi e funzionalità; Aggiunti sottocasi 11.1 e 11.2 dell'UC 11; aggiornata la sezione "Requisiti"	Sara Ristovic Marco Barbiero	Dario Pirrone
0.4.0	2026/04/30	Sviluppo sezioni "Casi d'uso" e Agenti	Sara Ristovic Marco Barbiero	Dario Pirrone
0.3.0	2026/04/15	Inizio scrittura della sezione "Requisiti"	Sara Ristovic	Marco Barbiero
0.2.0	2026/04/12	Scrittura della sezione 2	Dario Pirrone	Marco Barbiero
0.1.0	2026/04/08	Creazione documento e scrittura della sezione 1	Dario Pirrone	Marco Barbiero

Indice

1	Introduzione	5
1.1	Scopo del documento	5
1.2	Glossario	5
1.3	Riferimenti	5
1.3.1	Riferimenti normativi	5
1.3.2	Riferimenti informativi	5
2	Descrizione generale	5
2.1	Obiettivi del prodotto	5
2.2	Funzioni del prodotto	6
2.3	Caratteristiche degli utenti	7
2.4	Piattaforma di esecuzione [?]	7
3	<i>Casi d'Uso</i>	7
3.1	Introduzione	7
3.2	<i>Attori</i>	7
3.3	<i>Casi d'Uso</i> trasversali	7
3.3.1	UC 1: Visualizzazione report prodotto dall'agente	8
3.3.2	UC 2: Validazione umana di una proposta dell'agente	8
3.3.3	UC 3: Visualizzazione errore durante l'esecuzione dell'agente	9
3.3.4	UC 4: Esportazione del report	9
3.4	Elenco dei <i>Casi d'Uso</i>	10
4	Agente 1: Documentazione e <i>README</i>	10
4.1	Introduzione all'agente	10
4.2	Obiettivi e Funzionalità	10
4.3	<i>Casi d'Uso</i>	11
4.3.1	UC 5: Gestione del <i>README</i> di Progetto	11
4.3.2	UC 6: Generazione Documentazione Inline (<i>Docstring/JSDoc</i>)	11
4.3.3	UC 7: Generazione Documentazione <i>API</i>	12
4.4	Requisiti Funzionali	13
5	Agente 3: Scansione <i>Vulnerabilità OWASP</i>	13
5.1	Introduzione all'agente	13
5.2	Obiettivi e Funzionalità	13
5.3	<i>Casi d'Uso</i>	14
5.3.1	UC 8: Esecuzione Scansione <i>Vulnerabilità (OWASP)</i>	14
5.3.2	UC 9: Verifica Conformità "Policy-as-code" (CLAUDE.md)	15
5.3.3	UC 10: Generazione Suggerimenti di <i>Remediation</i>	16
5.3.4	UC 10.1: Generazione snippet di codice correttivo	16
5.3.5	UC 10.2: Generazione descrizione testuale della <i>Remediation</i>	17
6	Agente 4: <i>Changelog</i> da <i>User Stories</i>	18
6.1	Introduzione all'agente	18
6.2	Obiettivi e Funzionalità	18

6.3	<i>Casi d'Uso</i>	18
6.3.1	UC 11: Recupero e Filtraggio <i>User Stories</i>	18
6.3.2	UC 11.1: Rilevamento di task con metadati insufficienti	19
6.3.3	UC 11.2: Gestione delle ambiguità e delle incoerenze	20
6.3.4	UC 12: Generazione <i>Changelog</i> Tecnico (Dev-to-Dev)	20
6.3.5	UC 13: Generazione <i>Changelog</i> di Business (Dev-to-Client)	21
6.4	Requisiti Funzionali	22
7	Note Operative	22
8	Requisiti	22
8.1	Requisiti funzionali	23
8.2	Requisiti qualitativi	23
8.3	Requisiti di vincolo	24
8.4	Requisiti prestazionali [?]	24
8.5	Requisiti di sicurezza	24
8.6	Tracciamento	25
8.6.1	Fonte - Requisito	25
8.6.2	Requisito - Fonte	25
8.6.3	Riepilogo requisiti	25

1 Introduzione

1.1 Scopo del documento

Il presente documento, denominato **Analisi dei Requisiti_G**, ha l'obiettivo di esporre in modo dettagliato e non ambiguo i requisiti funzionali, qualitativi e di vincolo individuati per il progetto **Code Guardian_G**. Tali requisiti sono stati delineati a seguito di un'attenta analisi del capitolato e dei successivi incontri con il **Proponente_G**. Il documento si avvale dell'uso di diagrammi e descrizioni dei *Casi d'Uso_G* (UC) per illustrare le interazioni tra gli *Attori_G* esterni e il sistema, fornendo una base solida per le successive fasi di progettazione, codifica e verifica.

1.2 Glossario

Per evitare ambiguità terminologiche e garantire una comprensione univoca del documento, i termini tecnici, gli acronimi e le parole che assumono un significato specifico nel contesto del progetto sono riportati nel documento *Glossario*. Questi termini sono contrassegnati nel presente documento da una "G" a pedice ed è un collegamento ipertestuale che rimanda direttamente alla definizione corrispondente nel *Glossario*. Tali parole vengono contrassegnate solamente alla loro prima occorrenza in questo documento.

1.3 Riferimenti

1.3.1 Riferimenti normativi

- **Norme di Progetto_G** v1.0.0
- **Piano di Qualifica_G** v1.0.0
- Capitolato d'appalto C2 - **Code Guardian**
- Regolamento del Progetto Didattico

1.3.2 Riferimenti informativi

- Glossario v1.0.0
- Dispense **Analisi dei Requisiti**
- Dispense *Casi d'Uso*

2 Descrizione generale

2.1 Obiettivi del prodotto

L'obiettivo principale del progetto è lo sviluppo di un set di agenti intelligenti basati su modelli linguistici di grandi dimensioni (*LLM_G*), progettati per integrarsi in un'architettura di orchestrazione già esistente fornita dal **Proponente Var Group**. Il sistema,

denominato **Code Guardian**, mira ad automatizzare i processi di *Audit_G*, manutenzione e documentazione dei *Repository_G* software, riducendo il carico di lavoro manuale dei team di sviluppo e migliorando la qualità complessiva del codice. Nello specifico, il prodotto si prefigge di:

- **Garantire la coerenza documentale:** assicurando che il *README_G* di progetto e la documentazione tecnica (inline e *API_G*) siano costantemente allineati all'evoluzione funzionale della piattaforma.
- **Elevare gli standard di sicurezza:** introducendo il paradigma di *Policy-as-code_G*, attraverso il quale gli agenti verificano non solo le *Vulnerabilità_G* generiche *OWASP_G*, ma anche l'aderenza alle specifiche pratiche di sviluppo sicuro definite dal team.
- **Migliorare la trasparenza verso il business:** automatizzando la redazione di *Changelog_G* comprensibili al cliente finale, traducendo i progressi tecnici emersi dalle *User Stories_G* in valore di business tangibile.

2.2 Funzioni del prodotto

Il prodotto offre le sue funzionalità attraverso tre agenti specializzati coordinati dall'*Orchestratore_G* centrale:

1. Agente di Documentazione e *README*:

- **Manutenzione *README*:** creazione e aggiornamento dinamico del file *README* di progetto in base alle modifiche apportate alla piattaforma.
- **Gestione documentazione tecnica:** generazione e aggiornamento di documentazione inline (*Docstring_G*, *JSDoc_G*) e delle *API docs_G* per riflettere lo stato attuale del codice.

2. Agente di Security Review (*Policy-as-code*):

- **Analisi *Vulnerabilità*:** utilizzo delle *API* di *Claude Code_G* per l'identificazione di *Vulnerabilità* standard *OWASP*.
- **Verifica policy interne:** controllo dell'adozione di pratiche di sviluppo sicuro specifiche del team, utilizzando i file di configurazione *CLAUDE.md* presenti nel *Repository* come contesto operativo.

3. Agente di *Changelog* da *User Stories*:

- **Analisi *Sprint_G*:** lettura e filtraggio delle *User Stories* completate durante uno specifico *Sprint* di sviluppo.
- **Traduzione Business-Oriented:** conversione del linguaggio tecnico delle task in un linguaggio chiaro e comprensibile per il cliente, producendo *Changelog* sia tecnici che di business.

2.3 Caratteristiche degli utenti

Il sistema è progettato per inserirsi in un flusso di lavoro di sviluppo software moderno, interagendo con diverse figure professionali che operano all'interno di un team IT o nel rapporto con il cliente finale.

L'utente principale è rappresentato dallo **sviluppatore**, il quale interagisce con il sistema indirettamente attraverso il *Repository* *Git_G*. Egli beneficia dell'automatizzazione della documentazione tecnica e del feedback immediato sulla sicurezza, che gli permette di correggere eventuali *Vulnerabilità* o difformità rispetto alle policy aziendali in tempo reale, senza attendere revisioni manuali.

Una seconda figura chiave è il *Team Lead_G* o lo *Security Auditor_G*. Questi utenti utilizzano il sistema come strumento di monitoraggio e governance: definiscono le regole di "sviluppo sicuro" all'interno dei file *CLAUDE.md* e delegano all'agente di sicurezza il compito di verificarne l'adozione costante su tutto il codice prodotto.

Infine, il sistema si rivolge agli *Stakeholder_G* di business (come *Project Manager_G* o il cliente finale). Questi soggetti non necessitano di competenze tecniche approfondite, ma hanno bisogno di visibilità sull'andamento del progetto. Per loro, l'agente di *Changelog* funge da "traduttore", trasformando il gergo tecnico delle *User Stories* in report di valore che sintetizzano i progressi funzionali raggiunti al termine di ogni *Sprint*.

2.4 Piattaforma di esecuzione [?]

Il prodotto è concepito come un ecosistema di agenti integrati in un'infrastruttura già esistente, operante principalmente in ambiente cloud *AWS_G*. Essendo il sistema basato su interfacce di programmazione e automazione, l'esecuzione è garantita attraverso i principali ambienti di runtime come *Node.js_G* e *Python_G*. L'interazione con il codice sorgente avviene tramite la piattaforma *GitHub_G*, [sfruttando le *GitHub Actions_G* come principale motore di esecuzione per l'analisi e l'aggiornamento automatico dei *Repository*.

? NON E' OBBLIGATORIO]

3 *Casi d'Uso*

3.1 Introduzione

I *Casi d'Uso* sono costituiti da un diagramma *UML_G* e da una descrizione testuale che ne chiarisce il comportamento e gli obiettivi, al fine di definire in modo preciso le funzionalità che il prodotto deve offrire.

3.2 Attori

3.3 *Casi d'Uso* trasversali

Di seguito sono riportati i *Casi d'Uso* trasversali, ovvero validi per due o più agenti, mentre quelli specifici verranno esposti nelle sotto-sezioni apposite all'interno di ciascuna

sezione dedicata a ciascun agente.

3.3.1 UC 1: Visualizzazione report prodotto dall'agente

Lo Sviluppatore vuole consultare il risultato prodotto da un agente a seguito di un'analisi. Il sistema rende disponibili i report generati, permettendo allo Sviluppatore di visualizzare le informazioni principali relative all'esecuzione, come criticità rilevate, suggerimenti proposti e file coinvolti.

- **Attore Primario:** Sviluppatore.
- **Attori Secondari:** Orchestratore.
- **Trigger:** Lo Sviluppatore vuole consultare l'esito dell'analisi svolta da un agente.
- **Precondizioni:** L'Orchestratore ha eseguito almeno un agente. È disponibile almeno un report prodotto dall'agente.
- **Scenario Principale:**
 1. Lo Sviluppatore accede alla sezione dedicata ai risultati dell'analisi.
 2. Il sistema mostra la lista dei report disponibili.
 3. Lo Sviluppatore seleziona il report desiderato.
 4. Il sistema visualizza il contenuto del report, includendo eventuali criticità, suggerimenti e file coinvolti.
- **Scenari alternativi:** Se non sono presenti report, il sistema mostra un messaggio informativo.
- **Postcondizioni:** Lo Sviluppatore visualizza il report generato.

3.3.2 UC 2: Validazione umana di una proposta dell'agente

Ad oggi per ogni sistema agencico è necessario implementare *Human-in-the-loop_G* per assicurarsi i risultati affidabili.

- **Attore Primario:** Sviluppatore.
- **Attori Secondari:** *Repository Git*.
- **Trigger:** Lo Sviluppatore riceve una proposta generata da un agente.
- **Precondizioni:** Un agente ha prodotto una proposta di modifica. La proposta è disponibile sotto forma di *Pull Request_G*, *Patch_G* o suggerimento testuale [da decidere...?]
- **Scenario Principale:**
 1. Lo Sviluppatore apre la proposta generata dall'agente.

2. Lo Sviluppatore analizza il contenuto dalla proposta.
 3. Lo Sviluppatore decide se accettare, modificare o rifiutare la proposta.
 4. Il sistema registra l'esito della validazione.
- **Postcondizioni:** La proposta è stata accettata, modificata o rifiutata dallo Sviluppatore.

3.3.3 UC 3: Visualizzazione errore durante l'esecuzione dell'agente

Durante l'esecuzione di un agente può verificarsi un errore che impedisce il completamento dell'analisi o la generazione dell'output richiesto. In questo caso il sistema mostra allo Sviluppatore un messaggio esplicativo, indicando la causa dell'errore e, quando possibile, una possibile azione correttiva.

- **Attore Primario:** Sviluppatore.
- **Attori Secondari:** Orchestratore.
- **Trigger:** L'esecuzione di un agente termina con errore.
- **Precondizioni:** È stata avviata l'esecuzione di un agente. Si verifica un errore durante l'analisi o la generazione dell'output.
- **Scenario Principale:**
 1. L'agente incontra un errore durante l'esecuzione.
 2. L'**Orchestratore** riceve l'esito negativo.
 3. Il sistema mostra allo Sviluppatore un messaggio che descrive la causa dell'errore (es. repository non accessibile, API key non valida, file di configurazione assente, formato del report non generabile).
 4. Il sistema indica, se possibile, un'azione correttiva.
- **Postcondizioni:** Lo Sviluppatore visualizza un messaggio di errore esplicativo.

3.3.4 UC 4: Esportazione del report

Lo Sviluppatore vuole esportare un report generato da un agente per conservarlo, condividerlo o utilizzarlo nelle successive attività di sviluppo e verifica. Il sistema permette di selezionare il report desiderato e di salvarlo in uno dei formati disponibili.

- **Attore Primario:** Sviluppatore.
- **Trigger:** Lo Sviluppatore vuole esportare e salvare il risultato di un'analisi di un agente in forma di uno dei formati offerti.
- **Precondizioni:** È disponibile almeno un report generato da un agente.

- **Scenario Principale:**

1. Lo Sviluppatore seleziona un report.
2. Lo Sviluppatore sceglie il formato di esportazione.
3. Il sistema genera il file esportabile.
4. Lo Sviluppatore scarica o salva il report.

- **Postcondizioni:** Il report è stato esportato nel formato scelto.

- **Scenari alternativi:** Se l'esportazione fallisce, il sistema mostra un messaggio di errore.

3.4 Elenco dei *Casi d'Uso*

Per ogni agente preso in esame, verrà sviluppata una sezione dell'Analisi dei Requisiti dedicata, così da migliorare l'organizzazione del documento.

4 Agente 1: Documentazione e *README*

4.1 Introduzione all'agente

L'Agente di Documentazione e *README* rappresenta un pilastro fondamentale per la qualità del lavoro quotidiano degli sviluppatori e per la stabilità del progetto Code Guardian. Sappiamo bene come, nei cicli di sviluppo veloci, la documentazione sia spesso la prima a perdere il passo rispetto al codice, diventando presto superata. Questo agente nasce proprio per risolvere tale problema: il suo compito è automatizzare tutto il processo di aggiornamento. In questo modo, garantiamo che il file *README.md* e l'intera documentazione tecnica, sia quella interna al codice che quella dedicata alle *API*, siano sempre uno specchio fedele e puntuale dell'evoluzione del progetto, in qualsiasi momento.

4.2 Obiettivi e Funzionalità

L'agente si focalizza su tre aree critiche:

1. **Manutenzione del *README*:** Automazione della creazione e dell'aggiornamento del file principale di presentazione del *Repository*, adattandolo dinamicamente alle evoluzioni della piattaforma.
2. **Gestione della documentazione inline:** Analisi semantica del codice per la generazione di *Docstring* e *JSDoc*, assicurando che ogni modulo sia correttamente commentato.
3. **Documentazione delle *API*:** Estrazione automatizzata degli *Endpoint_G* per la creazione di guide tecniche per gli utilizzatori del servizio.

4.3 *Casi d'Uso*

Di seguito i *Casi d'Uso* dell'agente 1.

4.3.1 UC 5: Gestione del *README* di Progetto

Questo *Caso d'Uso* riguarda il mantenimento del file *README.md* principale per riflettere lo stato attuale del software.

- **Attore Primario:** Sviluppatore.
- **Attori Secondari:** *Orchestratore*, *Repository Git*.
- **Trigger:** Lo Sviluppatore effettua un push di modifiche strutturali o richiede l'aggiornamento tramite l'*Orchestratore*.
- **Precondizioni:** L'Agente 1 ha i permessi di lettura/scrittura sul *Repository*; esiste un file *README.md*.
- **Scenario Principale:**
 1. L'*Orchestratore* notifica l'Agente 1 della necessità di aggiornamento.
 2. L'Agente analizza file di configurazione (es. `package.json`) e struttura cartelle.
 3. L'Agente identifica nuove dipendenze o cambi di versione.
 4. L'Agente rigenera le sezioni interessate (es. guida all'installazione).
 5. L'Agente propone la modifica tramite *Pull Request*.
- **Postcondizioni:** È disponibile una proposta di aggiornamento del *README.md*, pronta per la validazione umana.
- **Estensioni:** UC 2 - Validazione umana di una proposta dell'agente.

4.3.2 UC 6: Generazione Documentazione Inline (*Docstring/JSDoc*)

L'agente interviene nel codice sorgente per garantire che ogni componente sia documentato correttamente.

- **Attore Primario:** Sviluppatore.
- **Attori Secondari:** *Orchestratore*.
- **Trigger:** Richiesta esplicita da parte dello Sviluppatore.
- **Precondizioni:**
 - L'Agente 1 ha i permessi di lettura/scrittura sul *Repository*.
 - Esiste un file di codice sorgente su cui eseguire l'operazione.

- (Opzionale) Esiste un file di configurazione per la documentazione online.

- **Scenario Principale:**

1. L'Agente individua funzioni o classi prive di documentazione o con documentazione obsoleta.
2. L'Agente interpreta la logica della funzione, i parametri di input e il valore di ritorno.
3. L'Agente genera una proposta di commento standardizzato (*JSDoc* per JS/TS o *Docstring* per *Python*).
4. Il sistema rende la proposta disponibile per validazione allo Sviluppatore.

- **Postcondizioni:** Lo Sviluppatore dispone di una proposta di documentazione inline da validare.

- **Sottocasi:**

- **UC 6.1:** Correzione di documentazione esistente non più coerente (es. parametri cambiati).
- **UC 6.2:** Segnalazione di logiche troppo complesse per la documentazione automatica.

4.3.3 UC 7: Generazione Documentazione *API*

L'agente estrae le definizioni degli *Endpoint* per creare una guida tecnica per i consumatori delle *API*.

- **Attore Primario:** Sviluppatore.
- **Attori Secondari:** Orchestratore.
- **Trigger:** Richiesta esplicita di generazione prima di un rilascio o rilevamento nuove rotte.
- **Precondizioni:** L'Agente 1 ha i permessi di lettura/scrittura sul *Repository*.
- **Scenario Principale:**
 1. L'Agente scansiona i file dei controller e i file delle *Rotte_G*.
 2. Estrazione di: metodo HTTP, URL, header, *Payload_G* richiesto e risposte possibili.
 3. Formattazione delle informazioni nel file *API.md* in formato *Markdown_G*.
 4. Salvataggio del documento nella cartella di documentazione predefinita.

- **Postcondizioni:** Disponibilità di un documento tecnico (*API.md*) aggiornato per gli *Stakeholder*.

4.4 Requisiti Funzionali

I requisiti funzionali per l'Agente 1 sono definiti per rispondere alle esigenze di coerenza documentale espresse dal proponente:

- **RF.1.1 (Aggiornamento *README*):** Il sistema deve identificare le modifiche strutturali nel *Repository* (es. aggiunta di moduli) e aggiornare le sezioni relative all'installazione nel *README.md*.
- **RF.1.2 (Standardizzazione Inline):** Il sistema deve analizzare le funzioni prive di documentazione e generare commenti standardizzati che descrivano parametri di input, output e comportamenti attesi.
- **RF.1.3 (Workflow di validazione):** Tutte le modifiche documentali proposte dall'agente devono essere sottomesse tramite *Pull Request*, subordinando l'applicazione delle stesse a una revisione umana obbligatoria per garantire l'accuratezza del contenuto.
- **RF.1.4 (Documentazione *API*):** Il sistema deve scandire i *controller* dell'applicazione per produrre o aggiornare una guida *Markdown* (es. *API.md*) che elenchi metodi HTTP, *Rotte* e *Payload* richiesti.

5 Agente 3: Scansione *Vulnerabilità OWASP*

5.1 Introduzione all'agente

L'Agente di Scansione Vulnerabilità OWASP ha l'obiettivo di supportare lo sviluppatore nell'individuazione di possibili *Vulnerabilità* di sicurezza presenti nel codice sorgente del *Repository*. L'analisi di sicurezza viene svolta facendo riferimento alle principali categorie di rischio definite da *OWASP* individuabili tramite analisi del codice, come: *Injection_G*, gestione non sicura dell'autenticazione, controllo degli accessi incompleto, gestione errata di dati sensibili, configurazioni insicure, ecc.

L'agente non garantisce la copertura esaustiva dell'intera *OWASP* Top 10. Il suo scopo è fornire una revisione automatizzata assistita, producendo segnalazioni, priorità e suggerimenti di *Remediation_G* che devono essere successivamente valutati da uno sviluppatore.

5.2 Obiettivi e Funzionalità

L'agente si concentra su tre aree principali:

1. **Analisi delle *Vulnerabilità OWASP*:** individuazione di pattern di codice potenzialmente riconducibili a *Vulnerabilità* note, come SQL *Injection*, uso insicuro

di funzioni crittografiche, esposizione di informazioni sensibili, *Hardcoded secrets_G*, ecc.

2. **Verifica delle policy interne di sicurezza:** controllo dell'aderenza del codice alle regole definite dal team nel file *CLAUDE.md*, secondo un approccio di tipo *Policy-as-code*.
3. **Generazione di remediation:** produzione di suggerimenti testuali o frammenti di codice [quale cosa??] correttivi per mitigare le vulnerabilità o le non conformità rilevate, senza applicare modifiche automatiche prima della validazione umana.

5.3 *Casi d'Uso*

Di seguito i *Casi d'Uso* dell'agente 3.

5.3.1 UC 8: Esecuzione Scansione *Vulnerabilità (OWASP)*

L'agente utilizza le *API* di *Claude Code* per identificare *Vulnerabilità* standard nel codice sorgente.

- **Attore Primario:** Sviluppatore.
- **Attori Secondari:** *Orchestratore*, *Repository Git*, *API Claude Code*.
- **Trigger:** Lo Sviluppatore richiede una scansione di sicurezza oppure l'*Orchestratore* avvia il processo dopo un *Commit_G* nel *Repository*.
- **Precondizioni:**
 - L'Agente 3 è correttamente configurato con le *API Key* di *Claude Code*.
 - Il codice sorgente è accessibile nel *Repository*.
- **Scenario Principale:**
 1. L'*Orchestratore* invia i file sorgente (o il diff dell'ultimo *Commit*) all'Agente 3.
 2. L'Agente 3 interroga le *API* di *Claude Code* inviando il codice per l'analisi.
 3. L'AI analizza il codice cercando pattern riconducibili alle *Vulnerabilità* della Top 10 *OWASP* (es. *SQL Injection*, *XSS_G*, *Hardcoded secrets*).
 4. L'Agente 3 riceve l'esito dell'analisi.
 5. L'Agente 3 associa ogni *Vulnerabilità* al file e, se disponibile, alla porzione di codice interessata.
 6. L'Agente 3 classifica ogni *Vulnerabilità* trovata nelle categorie di Top 10 *OWASP*.

7. L'Agente 3 formatta i risultati evidenziando la gravità (High, Medium, Low) di ogni falla trovata.
 8. L'Agente 3 inserisce le *Vulnerabilità*, la loro classificazione e le gravità nel report finale.
- **Postcondizioni:** Viene generato un report tecnico delle *Vulnerabilità* rilevate. Ogni *Vulnerabilità* rilevata è descritta con categoria, gravità e riferimento al codice interessato.
 - **Sottocasi:**
 - UC 8.1: Scansione specifica su un singolo file modificato. (include)
 - UC 8.2: Scansione completa dell'intero *Repository* (Full Audit). (include)
 - UC 8.3: Classificazione delle *Vulnerabilità* rilevate.
 - **Scenario Alternativo:** Se non vengono rilevate *Vulnerabilità*, l'Agente genera un report che indica l'assenza di criticità individuate.

5.3.2 UC 9: Verifica Conformità "*Policy-as-code*" (CLAUDE.md)

L'agente verifica che il codice rispetti le regole specifiche definite dal team nel file di configurazione.

- **Attore Primario:** Sviluppatore.
- **Attori Secondari:** **Orchestratore**, File di configurazione CLAUDE.md.
- **Trigger:** Avvio della revisione di sicurezza (manuale o automatica).
- **Precondizioni:** Nel *Repository* è presente un file denominato CLAUDE.md contenente le linee guida di sviluppo sicuro del team.
- **Scenario Principale:**
 1. L'Agente 3 legge il contenuto del file CLAUDE.md per estrarre il contesto e le regole (es. "usa sempre questa libreria per il logging", "non usare variabili globali").
 2. L'Agente analizza il codice sorgente confrontandolo con le direttive estratte.
 3. L'Agente identifica eventuali deviazioni dalle pratiche concordate dal team.
 4. L'Agente genera un elenco di violazioni della policy interna.
- **Postcondizioni:** Viene prodotto un elenco di non-conformità rispetto alle policy di progetto.
- **Sottocasi:**

- UC 9.1: Validazione del file CLAUDE.md (verifica che il file di configurazione sia scritto correttamente). (extend)

5.3.3 UC 10: Generazione Suggerimenti di *Remediation*

L'agente fornisce soluzioni pratiche per correggere i problemi di sicurezza o le violazioni di policy trovate.

- **Attore Primario:** Sviluppatore.
- **Attori Secondari:** *API Claude Code*.
- **Trigger:** Conclusione con esito positivo (rilevamento falle) degli UC 8 o UC 9.
- **Precondizioni:** Sono state identificate *Vulnerabilità* o violazioni di policy.
- **Scenario Principale:**
 1. Per ogni *Vulnerabilità* o violazione trovata, l'Agente 3 richiede a *Claude Code* una possibile correzione.
 2. L'AI genera uno snippet di codice "sicuro" che risolve il problema.
 3. L'Agente presenta allo sviluppatore il confronto tra il codice insicuro e la soluzione proposta.
- **Postcondizioni:** Lo sviluppatore riceve indicazioni chiare su come correggere il codice per renderlo conforme.
- **Inclusioni:**
 - UC 10.1: Generazione snippet di codice correttivo.
 - UC 10.2: Generazione descrizione testuale della remediation.
- **Estensioni:**
 - *Caso d'Uso* trasversale UC 2: Validazione umana di una proposta dell'agente.

5.3.4 UC 10.1: Generazione snippet di codice correttivo

Descrive la generazione di un frammento di codice alternativo per correggere una *Vulnerabilità* o una non conformità.

- **Attore Primario:** Sviluppatore.
- **Attori Secondari:** *API Claude Code*.
- **Trigger:** Una *Vulnerabilità* o non conformità può essere corretta tramite una modifica puntuale al codice..
- **Precondizioni:**

- La *Vulnerabilità* è sufficientemente circoscritta nel codice.
- L'Agente 3 può proporre una soluzione locale senza modifiche architetturali.

- **Scenario Principale:**

1. L'Agente 3 seleziona il frammento di codice vulnerabile.
2. L'Agente 3 richiede una versione corretta del frammento.
3. Il modello genera uno snippet alternativo.
4. L'Agente 3 presenta allo Sviluppatore il confronto tra codice originale e codice proposto.

- **Postcondizioni:** Lo Sviluppatore visualizza uno snippet di codice correttivo da valutare manualmente.
- **Scenari Alternativi:** Se la modifica non è localizzabile in uno snippet, l'agente rimanda a UC 10.2.

5.3.5 UC 10.2: Generazione descrizione testuale della *Remediation*

Descrive la generazione di una spiegazione testuale della correzione da effettuare.

- **Attore Primario:** Sviluppatore.
- **Attori Secondari:** *API Claude Code*.
- **Trigger:** Una *Vulnerabilità* o non conformità non può essere risolta in modo affidabile tramite uno snippet di codice.
- **Precondizioni:**
 - È disponibile una *Vulnerabilità* o non conformità rilevata.
 - L'agente dispone del contesto sufficiente per descrivere il problema.
- **Scenario Principale:**
 1. L'Agente 3 analizza il problema rilevato.
 2. L'Agente 3 genera una spiegazione della causa del problema.
 3. L'Agente 3 descrive le azioni consigliate per mitigare o correggere il problema.
 4. L'Agente 3 inserisce la descrizione nel report finale.
- **Postcondizioni:** Lo Sviluppatore riceve una descrizione chiara della *Remediation* consigliata.

6 Agente 4: *Changelog* da *User Stories*

6.1 Introduzione all'agente

L'Agente di *Changelog* da *User Stories* funge da mediatore semantico tra il progresso puramente tecnico del team di sviluppo e le necessità informative degli *Stakeholder* di business. In un contesto *Agile_G*, comprendere l'impatto reale di uno *Sprint* può essere complesso per figure non tecniche se i resoconti si limitano a liste di *Commit* o *Ticket_G* criptici. Questo agente automatizza la traduzione delle attività concluse in report chiari, garantendo che ogni aggiornamento del software sia accompagnato da una narrazione del valore di business generato, senza però tralasciare il dettaglio tecnico necessario alla manutenzione interna.

6.2 Obiettivi e Funzionalità

L'agente si concentra sulla sintesi e trasformazione del progresso di progetto attraverso tre direttrici:

1. **Recupero e Analisi dei Dati:** Integrazione con i sistemi di task management (Jira, *GitHub Issues_G*) per l'estrazione automatizzata dei task completati.
2. **Sintesi Semantica Multitarget:** Generazione di report differenziati; uno tecnico (*Changelog* Tecnico) per il team di sviluppo e uno di valore (*Changelog* di Business) per i clienti e il management.
3. **Validazione e Qualità del Contesto:** Monitoraggio della qualità informativa dei task per prevenire la generazione di report ambigui o privi di valore.

6.3 *Casi d'Uso*

Di seguito i *Casi d'Uso* dell'agente 4.

6.3.1 UC 11: Recupero e Filtraggio *User Stories*

Questo caso d'uso riguarda l'estrazione delle attività completate dai sistemi esterni per avviare il processo di analisi.

- **Attore Primario:** Sviluppatore / *Project Manager*.
- **Attori Secondari:** **Orchestratore**, Sistema di Task Management (*GitHub*/*Jira*).
- **Trigger:** Lo Sviluppatore richiede la generazione del report a conclusione di uno *Sprint*.
- **Precondizioni:**
 1. L'Agente 4 dispone delle credenziali e dei token di accesso validi per le *API* del sistema esterno.
 2. È stato specificato un identificativo valido per lo *Sprint* (*Sprint ID*).

3. Le task completate durante lo *Sprint* si trovano in uno stato terminale positivo.

- **Scenario Principale:**

1. Il *Project Manager* o lo Sviluppatore seleziona lo *Sprint* da analizzare comunicando lo *Sprint ID*.
2. Il sistema esterno restituisce la lista di tutte le lavorazioni associate.
3. L'Agente filtra la lista, mantenendo esclusivamente le *User Stories* e i task che si trovano in uno stato terminale positivo (es. **Done**, **Closed**).
4. L'Agente estrae da ogni task i campi fondamentali: Titolo, Descrizione, Criteri di Accettazione e Commenti rilevanti.
5. L'Agente mappa i dati in una struttura interna e li memorizza temporaneamente.

- **Inclusioni:**

- UC 11.1: Rilevamento di task con metadati insufficienti e segnalazione all'utente.
- UC 11.2: Gestione delle ambiguità se l'AI rileva discrepanze tra titolo e contenuto tecnico.

6.3.2 UC 11.1: Rilevamento di task con metadati insufficienti

- **Attore Primario:** Sviluppatore.

- **Attori Secondari:** **Orchestratore**, Sistema di Task Management.

- **Trigger:** Durante l'esecuzione del recupero (UC 11), l'agente individua una o più *User Stories* prive di descrizione o di criteri di accettazione minimi.

- **Precondizioni:** L'Agente ha superato la fase di autenticazione *API* e ha estratto la lista grezza delle task dello *Sprint*.

- **Scenario Principale:**

1. L'Agente analizza il contenuto testuale dei campi informativi di ogni task.
2. L'Agente rileva che una o più task presentano un livello informativo nullo o sotto la soglia minima stabilita dal sistema.
3. L'Agente contrassegna internamente queste task come "Non documentabili per scarsità di dati" ed esclude i relativi ID dal flusso di generazione automatica.
4. Il sistema genera un avviso nel log di esecuzione per informare l'utente delle specifiche task scartate.

- **Post-condizioni:** Il flusso di generazione dei report (UC 12 / UC 13) prosegue solo per le task complete di tutti i metadati, mentre l'utente riceve la notifica tempestiva dei dati mancanti da integrare manualmente nei task incompleti.

6.3.3 UC 11.2: Gestione delle ambiguità e delle incoerenze

- **Attore Primario:** Sviluppatore.
- **Attori Secondari:** **Orchestratore**, Modello AI (*LLM*).
- **Trigger:** Durante la validazione dei dati aggregati nell'UC 11, l'agente rileva una forte discrepanza semantica tra il titolo/descrizione della *User Story* e i dati tecnici correlati (es. log dei *Commit* associati).
- **Precondizioni:** I dati della *User Story* e i dati del *Repository Git* (*Commit*/branch) sono stati recuperati con successo e sono pronti per l'analisi.
- **Scenario Principale:**
 1. L'Agente esegue un confronto semantico automatico tramite *LLM* tra la documentazione della task e l'effettivo codice/messaggio di *Commit* associato.
 2. L'Agente identifica un conflitto (es. task registrata come "Modifica Testi" ma modifiche reali focalizzate sulla "Sicurezza delle API").
 3. L'Agente contrassegna la voce come "Ambigua" all'interno del sistema di orchestrazione.
 4. Il sistema presenta una notifica visiva allo Sviluppatore, richiedendo una risoluzione manuale o una conferma su quale contesto dare priorità (Documentale o Tecnico).
- **Post-condizioni:** Viene intercettato un potenziale errore di contesto prima della generazione del report definitivo, garantendo la perfetta coerenza tra ciò che viene dichiarato nel *Changelog* Tecnico e ciò che viene spiegato nel *Changelog* di Business.

6.3.4 UC 12: Generazione *Changelog* Tecnico (Dev-to-Dev)

- **Attore Primario:** Sviluppatore.
- **Attori Secondari:** Modello AI (*LLM*).
- **Trigger:** Conclusione con successo della fase di recupero, filtraggio e validazione dei dati dello *Sprint* (UC 11).
- **Precondizioni:**
 1. L'Agente 4 dispone del set di dati delle *User Stories* validato e privo di ambiguità bloccanti (superamento di UC 11.1 e UC 11.2).
 2. I parametri di configurazione del *Repository* sono accessibili.

- **Scenario Principale:**

1. L'Agente analizza il set di dati aggregato e raggruppa le lavorazioni in macro-categorie tecniche standard (es. **Added**, **Changed**, **Fixed**, **Deprecated**).
2. Per ogni *User Story*, l'Agente interroga l'*LLM* inviando i dettagli tecnici e i criteri di accettazione, richiedendo una sintesi in linguaggio ingegneristico asciutto e privo di ridondanze commerciali.
3. L'Agente genera la struttura del documento formattando l'output con un formato di *Markdown*.
4. L'Agente inserisce per ogni voce i collegamenti ipertestuali e gli identificativi (ID) corrispondenti ai *Ticket* di origine sul Sistema di Task Management.
5. La bozza del file prodotto viene memorizzata in attesa di essere validata.

- **Post-condizioni:** Viene generato un report tecnico pronto per la revisione umana (UC 2) o per l'esportazione diretta (UC 4).

6.3.5 UC 13: Generazione *Changelog* di Business (Dev-to-Client)

L'Agente esegue una traduzione semantica per trasformare il gergo tecnico in benefici tangibili per gli *Stakeholder*.

- **Attore Primario:** Sviluppatore.
- **Attori Secondari:** **Orchestratore**, Modello AI (*LLM*).
- **Trigger:** Conclusione con successo della generazione del *Changelog* tecnico previsto nell'UC 12.
- **Precondizioni:** Il file di *Changelog* Tecnico (UC 12) è stato generato correttamente.
- **Scenario Principale:**
 1. L'Agente riceve in input il testo del *Changelog* Tecnico prodotto dall'UC 12.
 2. L'Agente interroga l'*LLM* inviando il testo tecnico e applicando le direttive sul tono e sull'audience a cui si riferisce il documento.
 3. L'AI analizza le singole voci tecniche (es. modifiche a database, refactoring, patch di sicurezza) e ne estrae l'equivalente impatto funzionale.
 4. L'AI traduce i termini specialistici in benefici tangibili per l'utente finale, eliminando i dettagli implementativi di basso livello, poco rilevanti e poco comprensibili per il Cliente.
 5. L'Agente struttura la bozza del report di business e la memorizza, pronta per la validazione umana (UC 11).

- **Post-condizioni:** È disponibile un documento di *Changelog* orientato al business pronto per la revisione umana (UC 2) o per l'esportazione diretta (UC 4).

6.4 Requisiti Funzionali

I requisiti funzionali per l'Agente 4 definiscono le capacità di analisi e trasformazione dei dati richiesti:

- **RF.4.1 (Integrazione Sistemi Task):** Il sistema deve interfacciarsi con le *API* dei sistemi di Task Management esterni per il recupero dei metadati delle *User Stories*.
- **RF.4.2 (Generazione Multitarget):** Il sistema deve poter generare simultaneamente un report tecnico (orientato allo sviluppo) e un report di business (orientato al cliente).
- **RF.4.3 (Traduzione Semantica):** Il sistema deve utilizzare modelli di linguaggio (*LLM*) per mappare le attività tecniche a benefici funzionali descrittivi.
- **RF.4.4 (Validazione Dati):** Il sistema deve segnalare all'utente i task completati che presentano descrizioni ambigue o insufficienti per una corretta analisi automatizzata.

7 Note Operative

L'Agente 1 non è un software che richiede un intervento manuale costante. Esso opera in background: ogni volta che viene inviato nuovo codice al *Repository*, l'agente riceve una notifica automatica. Invece di rielaborare l'intero progetto, esamina solo le parti che sono state effettivamente modificate. Questo permette di mantenere la documentazione aggiornata senza rallentare il flusso di lavoro degli sviluppatori, che ricevono solo le proposte di modifica finali pronte per essere revisionate.

8 Requisiti

I requisiti riportati in questa sezione sono stati derivati dai *Casi d'Uso* descritti nel capitolo 3 e dalle indicazioni emerse dagli incontri con il **Proponente**. In particolare, il perimetro del progetto è stato chiarito come sviluppo di agenti specializzati da integrare in un'architettura già esistente, e non come realizzazione di una nuova architettura completa ad agenti.

8.1 Requisiti funzionali

Codice	Priorità	Descrizione	Fonti
RF.1	Obbligatorio	Il sistema deve aggiornare il file <i>README.md</i> del <i>Repository</i> in modo coerente con lo stato corrente del codice sorgente.	UC1
RF.2	Obbligatorio	Il sistema deve generare o aggiornare la documentazione inline del codice, come <i>Docstring</i> o <i>JSDoc</i> , per elementi sorgente supportati.	UC2
RF.3			
RF.4	Obbligatorio	Il sistema deve eseguire un'analisi di sicurezza del codice alla ricerca di <i>Vulnerabilità</i> riconducibili alle categorie <i>OWASP</i> rilevanti.	UC4
RF.4.1	Obbligatorio	Il sistema deve poter eseguire una scansione di sicurezza su un singolo file modificato.	UC4.1
RF.4.2	Obbligatorio	Il sistema deve poter eseguire una scansione di sicurezza sull'intero <i>Repository</i> .	UC4.2

8.2 Requisiti qualitativi

Codice	Priorità	Descrizione	Fonti
RQ.1	Obbligatorio	Gli output prodotti dagli agenti devono risultare utili al miglioramento del codice, ad es. tramite segnalazioni, suggerimenti, frammenti di codice o proposte di soluzione.	Verbale esterno
RQ.2	Obbligatorio	Devono essere definite modalità di test e validazione per gli agenti sviluppati.	Verbale esterno

8.3 Requisiti di vincolo

Codice	Priorità	Descrizione	Fonti
RV.1	Obbligatorio	Gli agenti sviluppati devono integrarsi in un'architettura già esistente fornita dal Proponente .	Verbale esterno
RV.2	Obbligatorio	Il progetto non deve prevedere l'addestramento di nuovi modelli, ma l'integrazione di modelli o servizi tramite <i>API</i> .	Verbale esterno
RV.3	Obbligatorio	Il sistema deve poter operare sui <i>Repository</i> di esempio e sul codice reso disponibile dal Proponente per integrazione e test.	Verbale esterno
RV.4	Obbligatorio	Il progetto non prevede lo sviluppo di agenti: Orchestratore , per aggiornamento librerie, per sostituzione del codice obsoleto e per mantenimento della copertura dei test, in quanto sono già in sviluppo presso Var Group.	Verbale esterno

8.4 Requisiti prestazionali [?]

8.5 Requisiti di sicurezza

Codice	Priorità	Descrizione	Fonti
RS.1	Obbligatorio	Le eventuali modifiche proposte al codice o ai file del <i>Repository</i> non devono essere applicate senza validazione umana preventiva.	Verbale esterno
RS.2	Obbligatorio	I risultati dell'analisi di sicurezza devono riportare almeno il tipo di problematica rilevata e una sua priorità o gravità.	UC4

8.6 Tracciamento

8.6.1 Fonte - Requisito

8.6.2 Requisito - Fonte

8.6.3 Riepilogo requisiti